Course Setup

Syllabus, git, Python, and getting your machine ready

Robert Pettis

University of South Carolina

Fall 2026

Today

1. What this course is, and how it is graded
2. Why we use Python, notebooks, and git
3. Git and GitHub: the three copies of everything
4. Installing it all, step by step
5. Staying in sync as I add material
6. How you hand work in
7. Your first notebook

The one thing to leave with

A working `cml` environment and your own fork of the course repository. Next class we start on the material itself.

Applied Econometric Methods and Data Science Basics with AI

- Tuesdays and Thursdays, 8:30 to 9:45 a.m.
- Instructor: Robert Pettis
- Office, office hours, and email: on the syllabus

The syllabus PDF is in the course repository. Everything on the next few slides is there in more detail.

What the course is about

Modern causal inference, and how machine learning fits into it.

One half

The prediction toolkit. Gradient descent, cross validation, regularization, trees, forests, neural networks, classification.

The other half

The causal framework. Potential outcomes, randomization, causal diagrams, regression and orthogonalization.

They come together in the second half of the term: heterogeneous treatment effects, meta-learners, double machine learning, causal forests.

By the end you should be able to estimate not only whether a treatment worked, but for whom and by how much, and to defend those estimates.

The shape of the term

Act 1. Identification

How to think causally.

- Causal inference
- Randomized experiments
- Graphical models
- Regression and orthogonalization

Act 2. Prediction

The machine learning the causal methods run on.

- Gradient descent
- Cross validation
- Ridge and lasso
- Trees and ensembles
- Neural networks
- Classification

Act 3. Causal ML

The two combined.

- Propensity scores
- Effect heterogeneity
- Meta-learners
- Double machine learning
- Causal forests

Act 2 is not a detour. Every method in Act 3 has a prediction machine inside it, and you cannot judge the causal estimate without understanding the machine.

Prerequisite

ECON 436 Introductory Econometrics, or an equivalent course in econometrics or statistics.

- Basic probability and regression are assumed.
- The Python you need is taught from the first week.
- No prior Python experience is required. That is what today is for.

If you have never programmed

You are in the right place and you are not behind. The first few weeks move slowly on the coding, and every lecture is a notebook you can rerun at your own pace.

Textbooks: required, and both free

- Matheus Facure, *Causal Inference for the Brave and True*. The main text.
  `matheusfacure.github.io/python-causality-handbook`
- James, Witten, Hastie, Tibshirani, and Taylor, *An Introduction to Statistical Learning with Applications in Python*. For the machine learning methods.
  `www.statlearning.com`

The free PDF of the second is identical to the print edition, so buy a physical copy if you prefer paper. You are not required to.

Textbooks: optional

- Facure, *Causal Inference in Python* (O'Reilly). The edited and expanded version of the main text. Worth buying if you want a polished print reference.
- Cunningham, *Causal Inference: The Mixtape*. Free online. A second edition, *The Remix*, is due August 25, 2026.
- Chernozhukov, Hansen, Kallus, Spindler, and Syrgkanis, *Applied Causal Inference Powered by ML and AI*. The most direct reference for the double machine learning part of the course. Free at `causalml-book.org`.
- Hastie, Tibshirani, and Friedman, *The Elements of Statistical Learning*. A deeper reference. Free online.
- Starmer, *The StatQuest Illustrated Guide to Machine Learning*, and the companion volume on neural networks. Friendly and highly visual, good for building intuition.

Lectures are live notebooks, not PDFs

I present from a Jupyter notebook. You have the same notebook open. You run the code yourself, change it, break it, and take notes in it.

- Bring a laptop if you have one.
- I add and revise notebooks all term, so you will pull down new material before most classes. The section on staying in sync shows how.
- You do not need to present anything from these notebooks. The slide styling is mine. Running the code is what matters for you.

Assessment

Item	Undergraduate	Graduate
Problem sets	20%	10%
Midterm tests	20%	10%
Final exam	15%	15%
Research project proposal	10%	15%
Research project report	25%	35%
Computational project	0%	15%
Participation	10%	0%

Grade scale

A	**B+**	**B**	**C+**
89.5 to 100	84.5 to 89.4	79.5 to 84.4	74.5 to 79.4
C	**D+**	**D**	**F**
69.5 to 74.4	64.5 to 69.4	59.5 to 64.4	below 59.5

Problem sets

At least three, each with some programming in Python.

- Submitted as a pull request from your own fork of the course repository. Setting that up is most of today.
- No late problem sets are accepted.
- You may use any package we have used in the course. Anything else needs permission first, and you will usually get it if you ask.

The package rule exists so that a problem set on writing a lasso by hand is not answered by calling somebody else's lasso. Ask if you are unsure.

Tests, final, and projects

- **Two in class midterm tests**, on the dates in the syllabus schedule.
- **Final exam**, cumulative, Tuesday December 8 at 9:00 a.m., set by the university examination schedule.
- Tests and the final are closed book, in class, with no AI and no electronic devices.

Research project

A prediction or causal inference problem investigated with methods from this course. A proposal of 1 to 3 pages covering question, data, and methodology. Then a report of at least 10 pages adding introduction, literature, results, and discussion. PhD students should use LaTeX.

Computational project (graduate only): build a Python package around a method from this course. Details early in the term.

ChatGPT, Claude, and Copilot are permitted here, and learning to use them well is part of your training. You will use them in research and in your careers, as I do.

The one firm condition

You are responsible for everything you submit. You must understand it, explain it in your own words, and reproduce it on your own.

You cannot audit what an AI produces without being able to do the work yourself. If you cannot derive the result or write the code unaided, you cannot tell when the tool is wrong, and it is often confidently wrong.

Generative AI: where the lines are

- **Research project.** Permitted and encouraged as a working tool. Disclose which tools you used and for what, and cite AI assisted text or code. You are responsible for the accuracy of everything in the report and must be able to defend it.
- **Problem sets.** You may use AI to help you learn. You must be able to reproduce and explain every step on your own.
- **Tests and final exam.** Closed book, in class, no AI, no electronic devices.

Under the USC Honor Code, using AI where it is not authorized, or submitting work you cannot explain, is an academic integrity violation. If you are unsure whether something is allowed, ask first.

Three tools, and why each one

Python

Where the causal machine learning actually lives. Every method in the second half of this course ships as a Python library first, and often only.

Jupyter notebooks

Code, output, math, and prose in one document. You can see the estimate and the code that produced it in the same place, which is how empirical work should be read.

Git and GitHub

How code is shared and versioned everywhere in the profession. It is how you will get my material and how you will hand in yours.

Git in one slide

Git takes **snapshots** of a folder over time.

- Every snapshot is called a **commit**, and carries a message saying what changed.
- Nothing is lost. You can go back to any snapshot.
- Two people can change the same project and git merges the changes.

Git is not GitHub

Git is the program on your laptop that takes the snapshots. **GitHub** is a website that stores a copy of them so other people can get at it. You could use git forever without GitHub. In this course we use both.

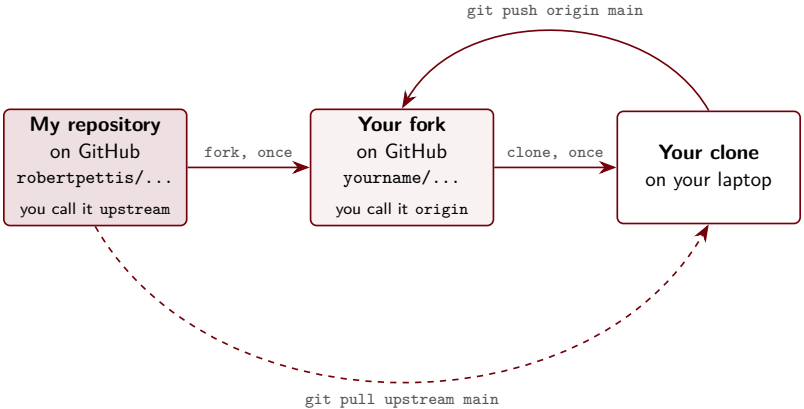
Why a fork and not just a clone

A **clone** is a copy on your laptop. A **fork** is a copy of the whole repository under your own GitHub account.

- You need somewhere to put your work that is yours and that I can see. That is the fork.
- You have no permission to write to my repository, and you should not want it. Forty students pushing to one repository is chaos.
- Your fork keeps a link back to mine, so pulling my updates stays easy all term.

This is exactly how open source works. You fork a project, work in your copy, and propose changes back. Nothing about this is specific to a classroom.

The three copies of everything



- You pull **from me** as I add material. You never push to me.
- You push **to your own fork**. That fork is what you submit.
- A **remote** is just a nickname for a repository somewhere else. `upstream` is mine, `origin` is yours.

The install, in one view

Budget about 30 minutes, most of it download time. Do it once.

1. Install Anaconda
2. Install Git
3. Tell git who you are
4. Make a GitHub account
5. Fork the course repository
6. Clone your fork
7. Add my repository as `upstream`
8. Create and activate the `cml` environment
9. Install the packages
10. Install JupyterLab and `pywrangling`
11. Check that it all worked
12. Launch

All of this is also written out in the README of the course repository, which you can read on GitHub in a browser before you have installed anything.

Step 1. Install Anaconda

Download the Anaconda Distribution installer:

`https://www.anaconda.com/download`

Run it and accept the defaults, with one exception:

Windows

Install for **Just Me**, and do **not** check "Add Anaconda to my PATH." It is offered, it is marked not recommended, and it causes conflicts later. You will use the Anaconda Prompt instead.

Already have Anaconda or Miniconda? Skip this step.

`https://git-scm.com/downloads`

Accept the defaults. On Windows this also installs Git Bash, which you can ignore.

Step 3. Tell git who you are

Git stamps a name and email on every snapshot. Set them once:

```
git config --global user.name "Your Name"
git config --global user.email "you@email.sc.edu"
git config --global --list
```

The last command echoes your settings back. Use your real name and your university email.

Step 4. A GitHub account

`https://github.com/signup`

- Free. Use your university email if you can.
- Pick a username you would not mind an employer seeing. This account tends to outlive the course.
- While you are there, the GitHub Student Developer Pack is free with a `.edu` address and worth having.

GitHub will ask you to set up two factor authentication. Do it now rather than at the moment you are trying to submit a problem set.

Step 5. Fork the course repository

In a browser, go to:

`https://github.com/Applied-Econometric-Methods/ECON594-Causal-Machine-Learning`

1. Click **Fork**, top right.
2. Leave the name as it is. Click **Create fork**.
3. You now have `github.com/YOURNAME/ECON594-Causal-Machine-Learning`.

Check

The page should say "forked from Applied-Econometric-Methods/ECON594-Causal-Machine-Learning" under the title. That line is the link that makes syncing work later.

Step 6. Clone your fork

Open a conda aware terminal:

- **Windows**: Start menu, then **Anaconda Prompt**. Not PowerShell, not Command Prompt.
- **macOS or Linux**: any terminal.

You should see `(base)` at the start of the prompt. Then:

```
cd path/to/wherever/you/keep/coursework
git clone https://github.com/YOURNAME/ECON594-Causal-Machine-Learning.git
cd ECON594-Causal-Machine-Learning
```

Replace `YOURNAME` with your GitHub username. Copy the URL off your own fork's page under the green **Code** button if you would rather not type it.

Step 7. Point at my repository too

Your clone knows about your fork, under the name `origin`. Give it my repository as well, under the name `upstream`:

```
git remote add upstream https://github.com/Applied-Econometric-Methods/ECON594-Causal-
    Machine-Learning.git
git remote -v
```

You should see four lines, two for `origin` pointing at your fork and two for `upstream` pointing at mine.

This is the step people skip

Without it, `git pull` has no idea where my new material lives, and in week 4 you will be downloading notebooks by hand.

Step 8. Create the environment

```
conda create -n cml python=3.11 -y
conda activate cml
```

Your prompt should now read `(cml)` instead of `(base)`.

Run `conda activate cml` every time you open a terminal for this course

Forgetting is the single most common source of "it worked yesterday." When something behaves strangely, look at your prompt first.

An environment is a private box of packages. Yours can hold pandas 2.3 while another project holds pandas 1.5, and neither disturbs the other.

Step 9. Install the packages

With `(cml)` active:

```
pip install numpy pandas scipy matplotlib seaborn scikit-learn statsmodels linearmodels
    networkx imageio-ffmpeg fklearn
```

PyTorch is a much larger download, so give it its own line and some patience:

```
pip install torch
```

That is the CPU build, which is all this course needs. No graphics card required.

Now the causal diagrams. This one is `conda`, not `pip`:

```
conda install -c conda-forge python-graphviz -y
```

Graphviz is a drawing program with a Python wrapper. `pip` gets you the wrapper only, and then diagrams fail with a confusing error about a missing `dot` program. If this step fights you, keep going. The notebooks fall back to plainer diagrams and nothing breaks.

Step 10. JupyterLab and the course toolkit

```
pip install jupyterlab ipykernel variable-explorer jupyterlab_cell_enhancements
```

- `ipykernel` is what actually runs your Python. JupyterLab does not install it for you, and without it Lab opens with no kernel.
- `variable-explorer` shows your variables and their values as you go, like Stata's or RStudio's.
- `jupyterlab_cell_enhancements` is quality of life in cells.

Then my own toolkit, which is not on PyPI, so pip takes it straight from GitHub. This is one reason you installed git:

```
pip install git+https://github.com/robertpettis/pywrangling.git
```

To pick up later updates, rerun that with `--upgrade --force-reinstall`.

Step 11. Check that it all worked

From inside the course folder, with `(cml)` active:

```
python check_setup.py
```

You want a run of `ok` lines and a closing `All set`. Anything marked `MISSING` names the package that did not install, so you can rerun just that piece.

Keep this command

It is also the thing to run when something breaks in week 9. Paste its entire output into an email to me and that is usually enough to diagnose the problem with no back and forth.

Step 12. Launch

From inside the course folder:

```
conda activate cml
jupyter lab
```

A browser tab opens. Because you launched from the activated environment, Lab uses that environment's Python, and the kernel shows as **Python 3 (ipykernel)** in the top right. That is the right one.

Start Lab from the course folder, not from your desktop

The notebooks read their data through paths like `./data/wage.csv`. Those only resolve if Lab was started from the right place.

The everyday git cycle

- Edit, snapshot, share. The same few commands, over and over.
- A **commit** saves on your laptop. A **push** puts it on GitHub. Until you push, only you can see it.
- It is your own copy: commit your notes, problem sets, experiments, not only what you hand in.

Stage and commit

Check what changed, mark what goes in the snapshot, then take it:

```
git status                  # what has changed
git add 02-my-notes.ipynb   # stage one file
git add .                   # or stage everything changed
git commit -m "Add notes from the regression lecture"
```

- **Staging** is you choosing what goes in each commit. Change ten files, commit only the three that belong together.
- `git status` is always safe and tells you where you stand. Run it constantly.

A good commit message

A note to your future self, and to me. Make it worth reading.

- One short summary line, present tense: “Add”, “Fix”, “Rewrite”
- Say what changed and why, not a line-by-line account
- One logical change per commit, so history stays readable

instead of	write
`stuff`	`Fix wage column name in problem set 1`
`update`	`Add cross validation to question 3`
`asdf`	`Rewrite gradient descent loop in vector form`

Pull and push

Committing saves your work on your laptop. Sharing it takes one more step.

```
git push origin main   # send your commits up to your fork
git pull origin main   # bring commits down from your fork
```

- **push** sends snapshots up to GitHub. **pull** brings them down. `main` is the default branch.
- A good rhythm: pull when you sit down, commit as you go, push when you stand up.
- Push often. A commit that lives only on your laptop is one spilled coffee from gone.

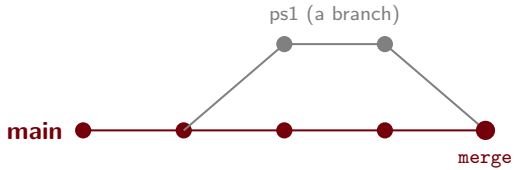
Branches

A **branch** is a separate line of work. Changes on one do not touch another until you merge.

- `main` is the branch you trust, the version that always works
- Branch off `main` to work without risking it:
  - `experiment`: try an idea you might throw away
  - `ps1`: one assignment, kept to itself
  - `fix-plot`: a single change you can review on its own
- `git checkout -b NAME` makes one and switches to it; `git checkout main` goes back

Nothing on a branch touches `main` until you choose to merge it.

Getting a branch back into main



- **Fast-forward:** `main` has not moved, so it slides up to your branch. Straight-line history.
- **Merge commit:** both moved, so git joins them with a commit that has two parents.
- **Pull request:** the same merge, reviewed on GitHub before it lands. How you merge into a repo you do not own, like mine.

Locally: `git checkout main` then `git merge ps1`. Across repos: open a pull request.

Pull requests

A **pull request** proposes merging one branch into another and asks the owner to accept it. A merge you request rather than do yourself.

- Used when you should not push straight to the target: someone else's repo, or a shared `main` a team protects
- The reviewer sees every change, comments on the exact lines, and merges or closes it
- Nothing lands until they accept, so you cannot break the target

In this course you open a pull request from your fork's branch into my `main`. That is how you hand work in.

.gitignore

A **.gitignore** file lists paths git should not track: scratch data, build output, secrets, notebook checkpoints.

- One pattern per line: `*.csv`, `__pycache__/`, `.ipynb_checkpoints/`
- Ignored files never appear in `git status` and are skipped by `git add .`
- Keeps commits to what matters, and keeps big or private files off GitHub

Expected a file to commit and it did not show up? Something may be ignoring it.
`git check-ignore -v THEFILE` names the rule.

I will be changing these files all semester

I add notebooks as we go, and I revise the ones already there: fixing errors, adding a section, rebuilding a figure.

So there are two things you need to be able to do, over and over:

1. Pull my new and updated material into your copy.
2. Push your own work up to your fork.

And one habit that makes both painless

Never edit my notebooks in place. Work in a copy. The next slides explain why, and it is worth taking seriously.

Getting my updates

Run this before most classes, from inside the course folder:

```
git pull upstream main
```

That fetches whatever I have changed and merges it into your copy. New notebooks appear, revised ones update.

Then push the result up to your own fork so it stays current too:

```
git push origin main
```

If the command line is fighting you

Your fork's page on GitHub has a **Sync fork** button that does the same thing in the browser. Click it, then run `git pull` to bring the result down to your laptop.

Why this works even though I change things constantly

I revise decks all term. You are working in the same folder. These do not collide, for one reason:

Git merges file by file

I edit files in `Lectures/`. You add files in your own folder. Two people editing different files is not a conflict, no matter how often either of us changes ours.

So a pull that arrives in the middle of your work updates my notebooks, leaves your files exactly as they were, and needs nothing from you.

The one way to break it is to edit my notebooks in place, which is what the previous slide is about.

The habit that avoids all the pain

Duplicate before you work

Before you touch `06-Cross-Validation.ipynb`, copy it to `06-Cross-Validation-MYNOTES.ipynb` and work in that.

- My original stays untouched, so `git pull` updates it cleanly forever.
- Your notes are a separate file that I never touch, so nothing of yours is ever overwritten.
- In class, run the original. Afterwards, work in your copy.

Notebooks are stored as one long machine readable file, and git merges them line by line. Two people editing the same notebook produces a mess that is genuinely unpleasant to untangle. One copied file avoids all of it.

When git refuses to pull

You will eventually see something like `Your local changes would be overwritten by merge.`

It means you edited a file that I also changed. First, see where you stand:

```
git status
```

If the edits it lists are ones you do not care about, throw them away and pull again:

```
git checkout -- 06-Cross-Validation.ipynb
git pull upstream main
```

If you do care, copy the file somewhere outside the course folder first, then do the same. Nothing here is an emergency, and no situation requires deleting everything and starting over.

Where your work lives

Everything you submit goes in one folder, named after your GitHub username:

```
submissions/YOURUSERNAME/
```

So problem set 1 from a student called `jsmith` is `submissions/jsmith/ps1.ipynb`.

- Nothing of yours ever goes in `Lectures/`. That folder is mine, and anything you put there collides with my next revision.
- Your folder is yours. I never write to it, so nothing you put there can be overwritten.

You hand work in with a pull request

A **pull request** proposes a change and asks the owner to take it. It is how every open source project on GitHub accepts work, and it is how this course accepts problem sets.

- You have no write access to my repository, so you cannot alter the course material. Nothing you do can break anything of mine.
- I read your code **in the pull request**, and leave comments on the specific lines they belong to. That is your feedback.
- The time you open it is the time you submitted. That is what I go by.

This is genuinely how professional work is reviewed. You are not learning a classroom procedure here, you are learning the workflow.

Submitting, step by step

Start from an up to date copy, then make a branch for this assignment:

```
git checkout main
git pull upstream main
git checkout -b ps1
```

Do the work in `submissions/YOURUSERNAME/`. Then snapshot it and send the branch to your fork:

```
git add .
git commit -m "Problem set 1"
git push origin ps1
```

A **branch** is a separate line of work. Keeping each problem set on its own branch means submitting one does not sweep up whatever you happen to be part way through on another.

Opening the pull request

After that push, GitHub prints a link, and your fork's page shows a **Compare and pull request** button. Either one gets you there.

1. Check the base is my repository and `main`, and the compare is your fork and `ps1`.
2. Title it with your name and the assignment.
3. Click **Create pull request**.

Check the Files changed tab before you click

It lists every file you are handing in. If it shows anything outside your own folder, fix that before submitting. If it is empty, you have not committed your work.

After you open it

- I review it, leave comments on the lines they refer to, and grade it.
- You will get an email for each comment. Reply in the pull request.
- I **close** it rather than merge it. Your work stays there permanently, with my comments, and the course folder stays clean.

Fixing something before the deadline

Commit and push to the same branch. The pull request updates itself. You do not open a second one.

Then, for the next assignment, start from a fresh branch off an updated `main`. That is the first three commands again.

The whole workflow on one slide

Before class

`git pull upstream main`	get my latest material

Handing in an assignment

`git checkout main`	start from the clean copy
`git pull upstream main`	make sure it is current
`git checkout -b ps1`	a branch for this assignment
`git add .`	mark your work for the snapshot
`git commit -m "..."`	take the snapshot
`git push origin ps1`	send the branch to your fork

then open the pull request on GitHub

Any time

`git status`	what has changed, and where you stand

When you are lost, `git status` is always safe to run and usually tells you what to do next.

The JupyterLab window

- **Left**: the file browser, showing the folder you launched from. Double click a notebook to open it.
- **Middle**: the notebook, a column of cells.
- **Top right**: the kernel indicator. The circle is hollow when idle and filled while your code is running.

The kernel

A Python session sitting behind the notebook, holding everything you have defined. Restarting it forgets every variable and starts clean. That is a fix, not a failure.

Cells

Two kinds matter.

- **Code** cells run Python and print their output underneath.
- **Markdown** cells hold text, headings, and math. This is where your notes go.

`Shift + Enter`	run this cell, move to the next
`Ctrl + Enter`	run this cell, stay here
`Esc` then `M`	turn the cell into markdown
`Esc` then `Y`	turn it back into code
`Esc` then `B`	new cell below

The thing that confuses everyone once

Cells run in the order **you** run them, not the order they sit on the screen.

The number in the brackets to the left of a cell, `In [7]`, is the order it was run in. If you run a cell, scroll up, change something, and run again, the notebook on your screen no longer matches the state of the kernel.

The honesty check

Kernel then **Restart Kernel and Run All Cells**. If it runs top to bottom without errors, your notebook is genuinely reproducible. Do this before you submit anything.

Install once, import every time

Two different things, and mixing them up is the most common early frustration.

Install

In the terminal. Once per environment. Downloads the package onto your machine.

```
pip install pandas
```

Import

In the notebook. Every session. Loads the package into this Python session.

```
import pandas as pd
```

`ModuleNotFoundError` means the package is not installed *in the environment you are running from*. Nine times out of ten the environment is the problem, not the package. Check your prompt reads `(cml)`.

Enough Python to start

```
import pandas as pd
import numpy as np

x = 5                        # a variable
names = ["Ann", "Bo"]        # a list

df = pd.read_csv("./data/wage.csv")   # a data frame
df.head()                    # first five rows
df.shape                     # (rows, columns)
df.describe()                # summary statistics
df["wage"].mean()            # one column, averaged
```

A **data frame** is a dataset with named columns. If you have used Stata, it is your data in memory. Everything in this course starts here.

Getting unstuck inside the notebook

- `pd.read_csv?` then run the cell shows the documentation.
- `Shift + Tab` with the cursor inside a function shows its arguments.
- Read the **last line** of an error message first. That is the part that says what went wrong. The rest is the trail of how it got there.

On the first cell of my notebooks

It imports `pywrangling` and applies my slide styling. It is there so the notebook looks right on the projector. Run it and forget it. You are not expected to present anything.

Troubleshooting

`jupyter: command not found` The environment is not active. Check your prompt, then `conda activate cml`.

`git: command not found` Git is not installed or not on your PATH. Reinstall and open a new terminal.

`ModuleNotFoundError` Almost always the wrong environment. Check the prompt reads `(cml)`, then rerun `python check_setup.py`.

Versions look wrong You are probably in `base`.

Diagrams look plain Graphviz did not install. Harmless. Rerun `conda install -c conda-forge python-graphviz -y` when you have a moment.

The nuclear option, which is fine

If you have been fighting the environment for an hour, delete it and rebuild. It takes five minutes and loses nothing:

```
conda deactivate
conda env remove -n cml
```

Then start again from Step 8.

Still stuck

Bring your laptop to office hours, or email me the exact command you ran and the complete error message. A pasted error usually takes under a minute to diagnose. Do not spend a weekend on an install.

Before next class

1. Anaconda and git installed.
2. GitHub account made, course repository forked, fork cloned.
3. `upstream` remote added.
4. `cml` environment created and packages installed.
5. `python check_setup.py` reads `All set.`
6. `jupyter lab` opens and a notebook runs.

If step 5 or 6 fails and you cannot see why, email me the output before the next class rather than the night before the first problem set.

Next time we start on causal inference.